PROJECT STATUS · 4 NOVEMBER 2026 · PLAIN-LANGUAGE VERSION

Do 3D models trust bad hints? Where we stand, and what comes next

What has been found, how the experiments are set up, what the planned fix does, why bad hints matter in the real world, and how the next GPU sessions decide the final paper.

The whole paper in three sentences

1. **The problem.** Programs that build 3D models from photos accept extra *hints* (where each camera was, lens settings, depth maps) and **obey them, even when they are wrong, and even when the program itself can tell**.
2. **It matters.** Real hint sources (AI depth estimates, phone compasses, GPS, long phone recordings) make errors of the size that hurts.
3. **A fix.** A cheap check that compares the hints with what the photos alone say can repair or drop bad hints without retraining, and leave good hints alone.

Part	Status
1. The problem	`shown` for MapAnything; partly for OmniVGGT; `next` π3-X
2. It matters in practice	`shown` for AI depth; `next` camera directions over whole rooms; error sizes from published studies
3. The fix	`built and tested on trial data`; final test after the next GPU session

1. The question

Programs like MapAnything turn a handful of photos into a 3D model. They can also take **hints**: where each camera was (position and direction), lens settings, and depth maps. Correct hints make them much better. Real hints come from phones, GPS, drones or AI depth estimators, and these are sometimes wrong. We ask three things:

1. Do wrong hints make the result **worse than using no hints at all**?
2. **Why** does it happen?
3. Can we **catch and fix** bad hints automatically, using only the photos, with no retraining?

2. What is done so far

Step	Question	Answer
E0	Does our pipeline work?	Yes. The setup checks passed, runs repeat bit for bit, and one 10-photo run takes 0.26 s.
E1	Do wrong hints hurt?	**Yes, badly, for camera-direction errors.** One camera turned 15° more than doubles the program's direction error compared with photos only (+114%, all 10 scenes). Slow drift: +142%. All cameras sharing one 5° compass error: +108%. Wrong camera *positions* are mostly shrugged off.
E2	What exactly happens, and does it happen elsewhere?	MapAnything **copies** wrong directions (77% of a 1° error, 96% of a 30° error) and never overrules them. It **knows** something is off: in 93 of 100 random pairs of one bad and one good camera, the bad one gets lower confidence. It obeys anyway. The damage happens inside the network, not in a simple averaging step. A second program (OmniVGGT) copies about half of a direction error. Real iPad hints **helped** both programs; **AI-guessed depth hurt both**. E1's simple filter catches one very wrong camera but throws away 12% of good cameras.

Step	Question	Answer
Now	Extended E2 and the first version of the fix (E3)	Laptop work done; GPU session prepared. Sections 3–8.

Source: `reports/e0_logs/8_det_*.txt`; `reports/e1_results.md`; `results/e1_eval/agg/signals.json`; `reports/e2_results/e2_results.pdf`

---

3. The setup

3.1 The programs (three)

Program	Role	Why it is included
MapAnything (Meta)	Main subject	Takes every kind of hint, camera by camera. This is where we found the problem.
OmniVGGT	Second program	Built differently (on VGGT). Shows whether the problem belongs to one program only.
π3-X (new)	Third program	Also takes all kinds of hints, camera by camera. A benchmark paper (SpatialBench) reports that it *partly overrules* wrong camera hints, the opposite of MapAnything, so it is a real test.
Depth Anything V2	Not tested itself	Only produces the **AI depth hints**, the way a phone without a depth sensor would.

3.2 The datasets (two, for different jobs)

Dataset	What it is	Why we use it	How
ETH3D	13 scenes, high-quality photos, laser-scanned exact ground truth	Exact truth lets us **break hints on purpose** by known amounts (turn one camera 15°, add drift, scale depth ×1.5) and measure the damage precisely	3 trial scenes, 10 final scenes; 10 photos per clip
SceneFun3D / ARKitScenes (iPad room scans)	Real iPad recordings with the iPad's **own** camera tracking (ARKit) and depth sensor (LiDAR), plus separately computed ground truth	Real-world hints with real mistakes	**Short** clips (E2-E, about 0.8 m of walking) and new **whole-room** clips (10 photos spread over the whole recording)

Why the whole-room clips? Over a short walk the iPad's tracking is almost perfect, so its hints helped. Across a whole room it drifts. Measured on 150 recordings:

10 photos spread over…	Typical size of the area	Worst camera's direction error (typical)	1 clip in 10 is worse than	Clips over 5°
a short stretch (E2-E style)	0.80 m	1.23°	2.71°	4.6%
the whole recording	3.36 m	2.88°	13.46°	20.7%

Photos alone get camera directions right to about 0.8°. So across a room, **the iPad's hint is often worse than the photos**, which is exactly where obeying should hurt.

Source: `results/e2x_survey/sf_device_survey.log`; `results/e2x_survey/sf_device_survey_clean.csv`; `reports/e2_results/e2_results.pdf (photos-only uncertainty)`

Data checked and set aside:

- **ADVIO** (phone): its "truth" comes from the same phone's motion sensor, so it cannot judge the phone.
- **Zurich drone:** GPS is typically 4.69 m off (worst 42.67 m) while the drone moves about 0.71 m per second. But the logged orientation is the drone's, not the camera's, which turns on its own. Kept as an option for GPS tests.
- **ScanNet++**: access not granted. **LaMAR**: access must be requested.

Source: `results/e2x_survey/advio_probe.log`; `results/e2x_survey/zurich_probe.log`

Quality control. The iPad dataset's "truth" is itself sometimes wrong: 493 of 73,512 frames jump away from their neighbours by more than 5° (80 by more than 30°), while the iPad's own tracking never jumps more than 30°. Every new clip therefore went through two checks (depth consistency, and whether the "true" poses fit the photos). **21 of 114 new clips failed and were set aside before any program saw them.**

Source: `results/e2x_survey/sf_gt_spikes.log`; `results/e2x_clip_check/`

3.3 How a test works

Each clip is 10 photos plus hints. Each program runs several ways on the **same** clip:

1. **Photos only**, no hints.
2. **Correct hints**, the best case.
3. **Wrong hints**: errors put in on purpose (ETH3D) or real iPad / AI hints (SceneFun3D).
4. **Wrong hints plus a remedy**: the existing remedies (3.4), and later our fix.

Errors are compared clip by clip, with the **room or scene** as the unit, so ten clips of one room cannot count as ten pieces of evidence. With many comparisons a statistical correction is applied, and an effect only counts if it is also **big enough to matter**: 1 percentage point for shape, 0.25° for direction, 5% for size.

What is measured: shape error of the 3D points; camera-direction error between camera pairs and the share of pairs within 5°; the standard pose score other papers report (AUC@30, now computed for 11,938 finished runs); depth error; real-world size error. For the fix also: how much damage it repairs, how often it wrongly touches good hints, and the extra running time.

Source: `results/e2c_eval/auc30.csv`; `results/e2df_eval/auc30.csv`; `results/e2d_eval/auc30.csv`; `results/e2e_ma_eval/auc30.csv`; `results/e2e_ov_eval/auc30.csv`

3.4 The existing remedies (what the fix must beat)

Name	What it does	What we know
Use all hints	Trust everything	This is the problem itself
R0 photos only	Ignore all hints	Safe, but throws away everything good hints add
R1 drop camera hints	Keep lens and depth, drop camera positions/directions	
R2 E1's filter	Compare each camera hint with the photos-only answer; drop it if it differs by more than 10° in direction or 15° in movement direction	Catches one very wrong camera, but drops 12% of good cameras and misses errors shared by all cameras
R3 confidence drop	Drop cameras the program itself is unsure about, then rerun	Same weakness on shared errors

Name	What it does	What we know
R4 classic geometry check	The traditional non-AI method (robust bundle adjustment)	Planned (CPU only)
ORACLE	Drop exactly the bad hints, using the truth	Cheating upper bound, for reference only
VPGGT, TCO	Methods from other papers	VPGGT: no public code. TCO: about 160 s per scene and no licence, so not run

4. Do bad hints really happen? (the question a reviewer will ask first)

If bad hints were rare, the paper would be about an exotic problem. So here is what is known about each real hint source: the size of its errors, and whether errors of that size hurt in our tests.

Hint source	Typical real error	Where the number comes from	Does that size hurt?
AI depth (cameras and phones without a depth sensor)	×1.42 too far, range ×1.13–×1.85, on **every** iPad clip	Measured by us (E2-E)	`yes, measured` shape +10.0 / +9.1 pp worse (MapAnything / OmniVGGT)
AI depth networks in general	34–55% average depth error on ETH3D	Metric3D v2 paper (TPAMI 2024), Table 3	depth ×1.5 hurts (E2-B)
Phone compass (camera direction)	indoors: 12.45° average, single readings 13–25°; outdoors after calibration 1–3°, 2–3× worse later; mean deviations 5.5–6.9°	Abadi et al. (IPIN 2013); Tartachynska & Pylypyak (2026)	one camera 15°, shared 5–10° and first camera 15° all hurt (E1, E2-B)
Phone GPS (camera position)	4.9 m open sky; 7–13 m in cities; up to 99.7 m. Our drone data: 4.69 m typical, 42.67 m worst	GPS.gov; Merry & Bettinger (PLOS ONE 2019); our Zurich survey	positions are mostly shrugged off; only big jumps hurt (E1)
Phone tracking over a whole room (ARKit)	worst camera 2.88° typical, over 5° in 20.7% of clips	Measured by us (150 recordings)	`being tested` (Part X-B)
Single-camera 3D tracking (monocular SLAM)	scale changed by 1 : 0.37 over one loop	Strasdat et al. (RSS 2010)	scale ×2 hurts (E2-B)
Lens data in photo files (EXIF)	"usually good, but can occasionally be off by more than a factor of two"	Snavely et al. (TOG 2006)	lens errors not yet shown to hurt
3D reconstruction software (COLMAP), often used to make hints	493 of 73,512 frames jump by more than 5° in a curated dataset	Measured by us (SceneFun3D)	same as the camera-direction rows

Source: `reports/e2_literature.md (literature sizes, with page numbers)`; `reports/e2_audit.md (E2-B: 7 error families harmful at literature-realistic sizes)`; `results/e2e_mono/`; `results/e2x_survey/`

Our answer to "isn't this rare?"

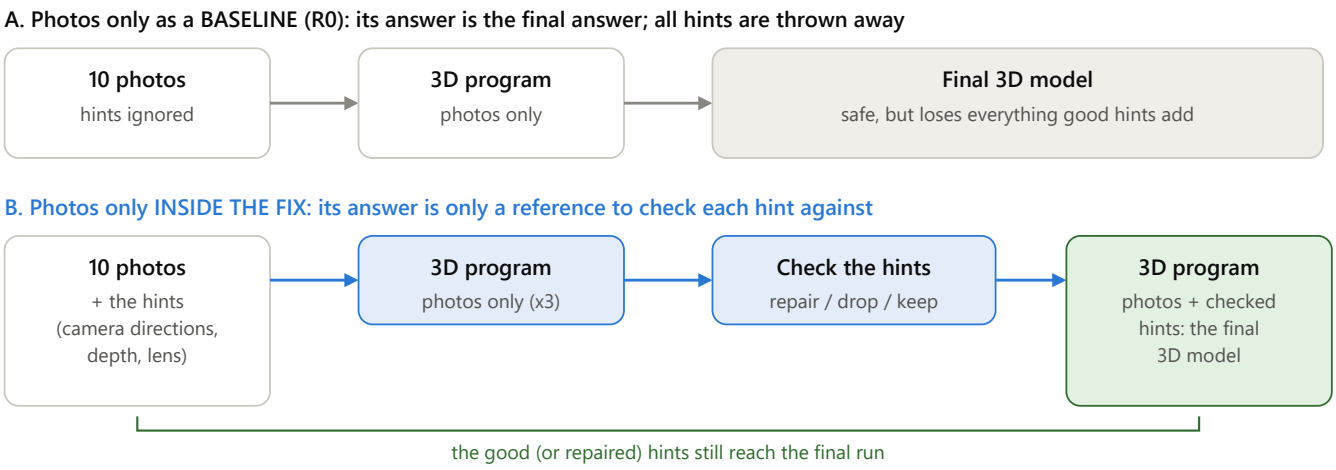
1. **Not for AI depth.** Every iPad clip had a scale error between 13% and 85%, and it hurt both programs.
2. **For camera directions it depends on the source**, and we say so openly: phone tracking over a short walk is good (it helped), while compass-based directions, GPS and long recordings are not.
3. **The fix is a seatbelt.** If it costs almost nothing on good hints and never makes them worse, it is worth having even when failures are uncommon. So the paper must show, and report, how rarely it touches good hints.

Honest gap: the only real-world harm we have *measured ourselves* so far is AI depth. Measured real-world harm from wrong camera directions depends on Part X-B (whole-room iPad clips).

5. The fix (E3), explained step by step

5.1 Why does the fix run "photos only", if that is a baseline?

The photos-only run has **two different jobs**, and only one of them is the baseline:



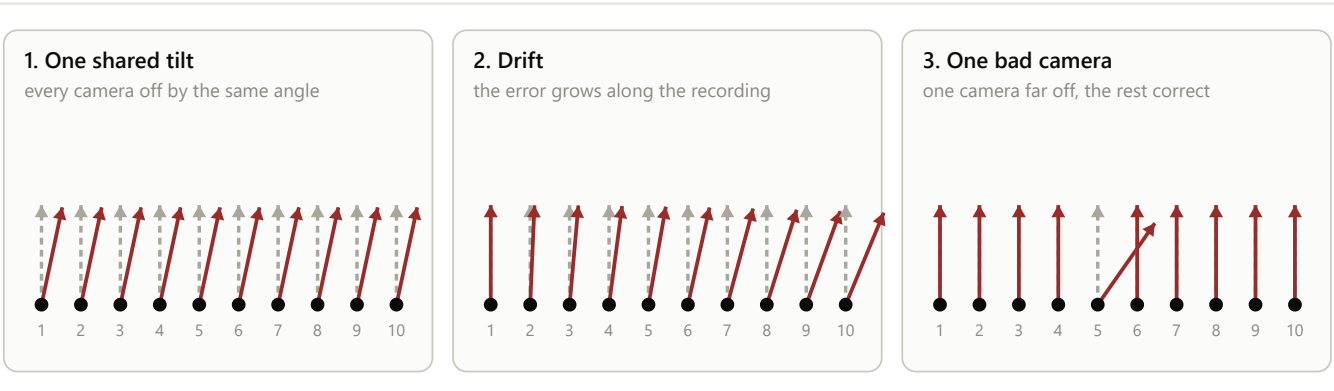
The reason the fix needs it: **no fixed choice wins everywhere.** With good hints, using them is much better (MapAnything's own paper, two-photo benchmark: relative point error 0.08 with photos only versus 0.02 with all hints). With bad hints, ignoring them is better (E1). A real system does not know which case it is in. So the fix decides **clip by clip and hint by hint**, and the photos-only answer is the **referee** it uses to decide, not the final answer.

	Use all hints	Photos only (R0)	The fix (goal)
Hints are good	best	clearly worse	as good as using all hints
Hints are bad	much worse (E1, E2)	safe	as good as photos only, or better (it keeps the good hints)

Source: `context.md section 3 (MapAnything paper, Table 2)`; `reports/e1_results.md`

5.2 The three ways hints go wrong

Picture 10 cameras along a walk. Each hint says which way a camera was pointing. Hints can be wrong in three typical ways:



- - - true direction — direction the hint gives (all cameras drawn pointing the same way; angles exaggerated so they are visible)

Kind	Plain example	What the photos-only answer shows
1. One shared tilt	The phone's compass is off by 5°, the camera is mounted slightly crooked, or the first camera's direction is wrong (every other camera is measured relative to it)	Every hint differs from the photos by the **same** angle. Each one looks only a little off; together they are consistently off.

Kind	Plain example	What the photos-only answer shows
2. Drift	Tracking adds a tiny mistake at every step: camera 2 is 0.5° off, camera 3 1°, … camera 10 4.5°	The difference **grows steadily** from camera to camera.
3. One bad camera	A GPS jump or a compass glitch next to metal	**One** camera disagrees strongly; the others agree.

5.3 What the fix does in each case

Kind	What the fix does	Why this way
One shared tilt	Measures the common angle and **turns all hints back** by it	The hints are still correct *relative to each other*, so all of that information is kept
Drift	Measures how fast the error grows and **undoes** it	Same: the rest of each hint is kept
One bad camera	**Drops that one camera's hint**	Its hint has nothing worth keeping
Depth too far or too near	Predicts depth from the camera hints alone, compares, and **rescales** the depth hints (e.g. divides AI depth by about 1.4)	AI depth is usually wrong in overall scale, not in shape

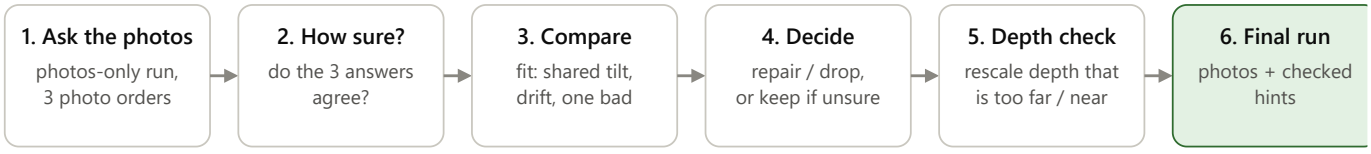
Worked example (shared tilt). The photos-only answer says camera 2 turned 10° and camera 3 turned 20° from camera 1. The hints say 15° and 25°. Every hint is 5° more than the photos, so the fix subtracts 5° from all hints. They now agree with the photos, and the final run uses **all 10 corrected hints**. E1's filter would have kept all of them unchanged, because each one is only 5° off, under its 10° limit, and the damage stays.

5.4 "Act only when the photos are sure"

The photos-only answer is not perfect either: typically 0.8° off (2.0° in the worst 10% of clips), and on some iPad trial clips 2–4.5° off overall, or 15° on a single camera, even when the hints were correct. If hints and photos disagree by 1°, we cannot tell who is wrong.

So the fix first measures **how sure the photos are on this clip**: it runs photos-only three times with the photos in a different order. If the three answers agree, the photos are a reliable witness here; if they wobble, they are not. A hint is repaired or dropped **only if it disagrees with the photos by several times the photos' own wobble**. Otherwise it is left alone.

Think of two witnesses. If one of them is unsure of what they saw, you do not use their story to overrule the other.



Extra cost: 3 photos-only runs per clip (once), plus 1-2 runs per hint set. One run takes about 0.26 s.

5.5 First trial numbers (trial clips only, not results)

Rows are the error put in on purpose. "Fix measured" is what the fix's comparison found, without being told.

Error put in	Fix measured	Same check with correct hints
One camera 15°	15.0° on that camera	under 1° on the others
First camera 15°	14.8° shared	0.6°
Shared 5° compass error	4.7°	0.6°
Drift levels 1 / 2 / 3	4.2 / 7.7 / 15.1°	0.9°

Source: `results/e2x_survey/e3_witness_dev.csv (3 ETH3D trial clips and 8 iPad trial clips; first prototype)`

> **The open risk.** On the iPad trial clips the photos themselves were sometimes unreliable (above). If the "how sure" measurement does not catch those cases, the fix would damage good hints. This is exactly what the calibration in the next GPU session checks first.

---

6. Extended E2: what the next GPU session does

Part X-A: is obeying common?

π3-X gets exactly the same broken ETH3D hints as MapAnything and OmniVGGT (identical down to the random numbers). Does it copy wrong hints, or overrule them as SpatialBench suggests? Either answer is informative.

Part X-B: do real hints hurt when the error is bigger?

All three programs on the whole-room iPad clips (π3-X also on the short clips): photos only, true hints, iPad poses, iPad depth sensor, AI depth, and remedies R2/R3. Do real iPad poses still help across a whole room? Does AI depth hurt again?

Part E3-calibration: setting the fix's cut-offs without peeking

- The fix runs on **calibration rooms** (10 rooms never used for final results) and the ETH3D **trial** clips, with every kind of broken hint and with real hints. Every decision it makes is logged.
- **Rule, fixed in advance:** choose the cut-off so that, **on correct hints**, the fix acts on at most 5% of clips (shared/drift repairs) and drops at most 2% of cameras. "Do no harm" comes first.
- How well it catches errors is recorded but **not** used to choose the cut-off, so the choice cannot be tuned toward a nice-looking result.

Jobs run most important first, so an early stop only loses the least important ones. Each final run starts only if its trial run passes automatic checks. Estimate (not measured): about 2.5–3.5 GPU-hours; π3-X's speed is unknown until its trial run.

Source: `experiments/E2X.md (sections 4-6); configs/e2x_*.json; scripts/session_queue_e2x.sh`

---

7. How E3 gets decided

1. **After the GPU session** (laptop): analyse X-A, X-B and the calibration.
2. **Lock the E3 plan** before the final E3 run: the cut-offs from calibration, the final clips, the remedies to beat, and the success rules.
3. **Second GPU session:** the fix against all remedies on the **final** clips: ETH3D final scenes with every kind of broken hint, whole-room iPad clips with real iPad hints, iPad clips with AI depth.

Proposed success rules (fixed in step 2, so not final yet):

- **Repairs damage:** on errors shared by all cameras (compass, drift, first camera, depth scale), where the existing remedies leave at least half of the damage, the fix removes clearly more. The original target was about 70% of what the cheating ORACLE removes.

- **Does no harm:** on correct hints and on good real iPad hints, any change stays below the "big enough to matter" thresholds.
- **Beats the best existing remedy** on those error types.

Outcome	Paper
π3-X also obeys; whole-room iPad hints or AI depth hurt; the fix passes	**Strongest workshop paper:** a general problem, real-world evidence and a working fix
The fix passes for some error types only (e.g. depth scale and shared errors)	Paper scoped to those types, stated honestly
The fix touches good hints too often, even at strict cut-offs	Analysis paper (problem + mechanism + why simple fixes fail); the fix becomes future work

---

8. One story, not a mix of everything

Every experiment answers one of three questions; anything that does not goes to the appendix or is dropped.

The paper's question	Main paper	Appendix	Not in the paper
1. Do the programs obey wrong hints?	Trust curves: how much of a wrong hint is copied; "knows but obeys" (confidence); three programs side by side	The E1 audit; the averaging (mean vs median) check showing the damage is inside the network; 4- and 20-photo clips; partial hints	—
2. Does it matter in practice?	Real error sizes of each hint source (section 4); AI depth harm; whole-room iPad hints	Ground-truth quality checks and excluded clips	ADVIO; Zurich drone (unless a GPS test is added)
3. Can it be fixed without retraining?	The fix vs all remedies; harm on good hints; extra time	Settings of the fix; the drop-only variant	—

> **Figure 1 of the paper (planned):** three approaches (use all hints, photos only, the fix) on good hints and on bad hints. Using all hints wins on good hints and loses badly on bad ones; photos only is the reverse; the fix aims to be close to the best in both. That one picture is the whole paper.

---

9. What is needed next

1. Approval of the plan in `experiments/E2X.md` (section 5), so it can be locked.
2. The RTX PRO 6000 VM for one session (estimate 2.5–3.5 GPU-hours).
3. A LaMAR access request (lamar.ethz.ch), which only the project owner can submit.